CVM++

Runtime flow, data structures, opcodes, and execution traces

What happens from ./build/cvmpp script.cvm to printed output

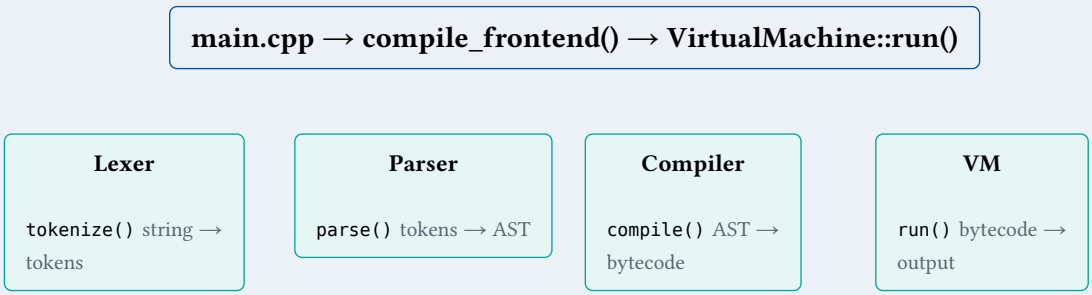
Introduction

This guide describes **how CVM++ runs internally**: entry points, each pipeline stage, bytecode layout, the VM loop, opcode semantics, and two worked traces. For setup and glossary, see the **Project and Build** guide.

Master pipeline

When you run `./build/cvmpp examples/hello.cvm`:

Execution pipeline



Orchestration lives in `src/compile.cpp`:

```
FrontEndResult compile_frontend(std::string source);
// 1. Lexer::tokenize()
// 2. Parser::parse()
// 3. Compiler::compile()
// main.cpp then calls execute() on the BytecodeChunk
```

Entry points

Mode	Command	Behavior
File	`cvmpp script.cvm`	Read file, compile, run; fresh globals
REPL	`cvmpp`	Interactive lines; `VmSession` keeps globals
Debug	`-d`	Token table + AST + bytecode disassembly
Compile-only	`-c`	Stop after bytecode generation
Quiet	`-q`	Suppress status banners

Stage 1 — Lexer

Input: full source string. **Output:** `vector<Token>` + diagnostics.

The scanner walks characters left-to-right:

- Skips whitespace and `//` comments
- Reads integers with overflow checking
- Recognizes keywords: `let`, `fn`, `return`, `if`, `else`, `while`, `print`, `input`
- Emits multi-character operators: `==`, `!=`, `<=`, `>=`
- Ends with `Eof`

Failure phase: `LEXER` — invalid character, overflow, null byte.

Stage 2 — Parser

Input: tokens. **Output:** `Program` AST.

```
Program
 ├── functions[]     top-level fn declarations
 └── statements[]    main script body
```

Expression precedence (low to high): equality (==, !=) → comparison (<, >, <=, >=) → +/- → */// → unary - → primary (literal, variable, call, `input`, parentheses).

Failure phase: `PARSER` — unexpected token, bad assignment target, unbalanced braces. Uses panic-mode `synchronize()` recovery.

Stage 3 — Compiler

Bytecode layout

> **Note**
>
> | | |
> |---|---|
> | **Offset 0** | `JUMP` skips function bodies |
> | **Functions** | `LOAD_LOCAL`, `RETURN`, … |
> | **Main entry** | Top-level statements |
> | **End** | `HALT` |

Without the entry jump, the VM would start inside a function and fail on `LOAD_LOCAL`.

Control-flow codegen

Construct	Technique
`if` / `else`	`JUMP_IF_FALSE`, forward `JUMP`, patch offsets
`while`	Loop label, condition, body, jump back
Variables	Intern name → `LOAD_VAR` / `STORE_VAR` index
Locals	Parameter slots → `LOAD_LOCAL` / `STORE_LOCAL`
Calls	Push args left-to-right, `CALL fn_index argc`

Stage 4 — Virtual machine

Runtime state

Field	Purpose
`ip_`	Instruction pointer into `chunk.code`
`stack_`	Operand stack for expressions

`globals_`	Indexed globals in file mode
`frames_`	Call stack: return IP + locals
`session_`	Optional REPL `unordered_map` by name

Execution loop

```
while ip in bounds and no error:
    if steps > 1_000_000: error (infinite loop guard)
    opcode = read_u8()
    dispatch opcode
```

Failsafes

Check	Result
Stack depth > 65536	Stack overflow error
Pop empty stack	Underflow error
Divide by zero	Runtime error, exit 2
Uninitialized global	Load error with hint
`RETURN` outside call	VM error
Bad jump target	IP overrun error

Opcode reference

Stack notation: **a, b** = pop order (b on top). ⟶ pushes result.

Opcode	Operands	Effect
`PUSH_INT`	i64	⟶ integer
`PUSH_BOOL`	u8	⟶ boolean
`LOAD_VAR`	u16 index	⟶ global
`STORE_VAR`	u16 index	pop ⟶ global
`LOAD_LOCAL`	u8 slot	⟶ local
`STORE_LOCAL`	u8 slot	pop ⟶ local
`ADD...DIV`	—	pop b, a ⟶ result
`EQ...GE`	—	pop b, a ⟶ bool
`INPUT`	—	⟶ stdin value
`PRINT`	—	pop ⟶ output buffer
`JUMP`	u32 offset	set IP

`JUMP_IF_FALSE`	u32 offset	pop; jump if false
`CALL`	u16 fn, u8 argc	new frame; jump to entry
`RETURN`	—	pop value; restore frame
`HALT`	—	stop

CALL and RETURN

> **Function call sequence**
>
> 1. Caller pushes arguments onto the stack.
> 2. `CALL` pops `argc` values into `frame.locals`, saves `return_ip`, sets `ip` to function entry address.
> 3. Function body runs (`LOAD_LOCAL`, arithmetic, nested calls).
> 4. `RETURN` pops return value, restores `ip` from frame, pushes result for caller.

Trace A — `print 1 + 2;`

Step	Instruction	Stack after
1	`PUSH_INT 1`	`[1]`
2	`PUSH_INT 2`	`[1, 2]`
3	`ADD`	`[3]`
4	`PRINT`	`[]` — prints 3
5	`HALT`	done

Trace B — `print factorial(5);`

```
fn factorial(n) {
  if (n <= 1) { return 1; }
  return n * factorial(n - 1);
}
print factorial(5);
```

Step	What happens
1	Main emits `CALL factorial` with argument `5`
2	Frame: `locals[0]=5`, IP at function entry
3	Recursive calls until `n <= 1`, each `RETURN` passes value back
4	Final result `120` on stack after outermost return

5	`PRINT` outputs `120`

REPL vs file mode

Aspect	File	REPL
`VmSession`	`nullptr`	Active — globals persist
Storage	Indexed `globals_` array	`unordered_map` by name
Between runs	Reset	Keeps variables

Debug workflow

```
./build/cvmpp -d examples/hello.cvm
```

Read output in order: **tokens → AST → bytecode → runtime lines**.

For bytecode only in the REPL: `:disasm examples/hello.cvm`

Source dependencies

```
main.cpp → compile.hpp → lexer, parser, compiler, vm, ui
compile.cpp → lexer → token, diagnostic
            → parser → ast
            → compiler → bytecode, opcode
            → vm → value
```